

Phase 2 - Agent B: Educator Global Mental Model & Storyline

目标参数: `lecture_01`

输入文件:

- `E:\allwork\cs336\lecture\lecture_01.md`
- `output/cs336/lecture_01/01_Phase1_Architect.md`

输出文件: `output/cs336/lecture_01/02_Phase2_Educator.md`

1. The Core Conflict

本讲的核心冲突是：语言模型已经工业化到个人无法复刻，但真正的研究理解又必须回到系统底层。

讲义开头先指出一个断裂：2016 年的研究者会自己实现和训练模型；2018 年的研究者下载 BERT 之类的模型并 fine-tune；今天很多研究者直接 prompt API 模型。抽象层级提高了生产力，但语言模型的抽象是 leaky abstraction。你不知道数据、架构、训练细节、系统瓶颈时，就很难判断一个现象到底来自模型能力、数据污染、训练 recipe、tokenizer、推理系统，还是规模本身。

这就是 CS336 的动机：不追求复刻 frontier model，而是把可迁移的东西拆出来。讲义明确区分三类知识：

知识类型	是否容易迁移到 frontier model	本课处理方式
Mechanics	高	从 scratch 实现
Mindset	高	始终围绕资源效率
Intuitions	部分	承认很多经验来自实验

随后，课程把这个冲突压缩成一个工程问题：给定资源预算，怎样训练出最好的模型？这就是 Phase 1 中的核心公式：

accuracy = efficiency x resources

tokenization 是这个冲突的第一个小型战场。我们当然可以把文本当 byte 处理，因为 byte 干净、固定、可逆；但这样序列太长，后面的 Transformer 会为每个 byte 付出计算成本。我们也可以按 word 切，但词表会爆炸，新词和稀有词会拖垮系统。BPE 被引入，是因为它在这两个极端之间找到一个工程上可接受的折中。

2. The Global Mental Model

整门课可以理解为一条高成本的文本炼油生产线，而 tokenizer 是第一台切料机。

想象你有一座炼油厂，输入不是原油，而是互联网上的 raw text。你的最终目标不是“处理文本”，而是把文本转化成能让模型学习的稳定燃料。整条生产线每前进一步都要付费：搬运数据要带宽，存中间结果要内存，训练参数要 FLOPs，跨机器同步要通信。

在这条生产线上：

- Tokenizer 是切料机：把原始字符串切成标准零件。

- Transformer 是反应炉：让这些零件相互作用并形成预测能力。
- Optimizer 和 learning-rate schedule 是控制系统：让反应稳定，不爆炸也不停滞。
- Kernels 和 parallelism 是管道与泵：减少数据搬运，让 GPU 真正做计算。
- Scaling laws 是小试装置：用低成本实验预测大规模生产结果。
- Data curation 是原料筛选：避免把昂贵算力浪费在坏料上。
- Alignment 是精炼工序：让已具备能力的模型朝人类偏好和任务目标调整。

tokenization 的关键在于切料尺度。如果切得太碎，每个 byte 都是一块零件，后续反应炉要处理的 item 数暴涨。如果切得太粗，每个 word 都是一种零件，仓库目录会爆炸，而且一遇到新零件就失效。BPE 的行为像一台会根据历史订单自动调整刀模的切料机：常见组合被合并成大零件，罕见组合保留为小零件。

这个隐喻能解释为什么本讲虽然从课程介绍开始，却最终落到 tokenizer：CS336 的所有技术选择都不是孤立技巧，而是在同一条资源受限生产线中做瓶颈管理。

3. The "Aha!" Moment

本讲最反直觉的技巧是：不要先问“语言学上正确的 token 是什么”，而要问“什么表示能在硬件预算下最有效地保真压缩”。

初学者很容易以为 tokenizer 是一个语言学问题：英文按空格切，中文按词切，标点单独处理。但这条路会很快遇到工程现实：词表不固定、稀有词太多、未登录词需要 UNK，而 UNK 会把原始信息直接抹掉。

BPE 的聪明之处是把问题改写成数据压缩问题：

从 byte 开始，保证任何字符串可表示；
再反复合并最常见的相邻 pair，保证常见模式更便宜。

这个做法同时抓住三个目标：

目标	BPE 如何满足
可逆性	从 UTF-8 bytes 出发，decode 时拼回 bytes
固定词表	初始 256 bytes，加上有限次 merge
更短序列	高频 byte 序列被合并成单 token

真正的 aha 不在于 merge pair 这个操作有多复杂，而在于它把文本表示从人工规则变成了可训练的系统组件。tokenizer 不再是模型外面一个无关紧要的预处理脚本，而是会改变 seq_len、vocab_size、attention 成本、embedding 成本和训练吞吐的工程决策。

4. Storyline Reconstruction

本讲的叙事线是：先解释为什么必须从底层理解语言模型，再用 tokenization 展示这种底层理解的具体形态。

第一段讲课程存在的理由。frontier model 太贵、太封闭，无法作为课堂里的实验对象。但研究者仍需要理解模型内部机制，否则只能在 API 抽象层上猜测。

第二段讲语言模型的发展史。Shannon、n-gram、LSTM、seq2seq、attention、Transformer、GPT-2、scaling laws、GPT-3、Chinchilla、Llama、DeepSeek、Qwen 等内容构成一条线：语言模型进步不是单靠“更大”，而是靠能随规模增长仍然有效的算法、数据和系统工程。

第三段讲课程结构。basics、systems、scaling laws、data、alignment 五个单元不是并列主题列表，而是从“能训练一个模型”逐步推进到“能高效训练、预测扩展、选好数据、对齐行为”的完整工程闭环。

第四段进入 tokenization。这里第一次把抽象目标落成代码接口：`encode` 和 `decode`。character、byte、word 三种方案依次失败，暴露出词表大小、序列长度、稀有项、OOV 和 compression ratio 的冲突。BPE 在这些冲突中给出实用折中。

因此，`lecture_01` 的教学作用是给学生建立一个判断标准：以后每看到一个模型技巧，都要问它解决了哪种资源瓶颈，以及它把成本转移到了哪里。

5. Phase 3 Initialization State

Phase 3 已具备上下文：后续问题应保持 Agent A/B 分工，先诊断数学、形状或资源问题，再转化为教学引导。

已生成的两个产物：

- `output/cs336/lecture_01/01_Phase1_Architect.md`：数学、接口、张量和工程骨架。
- `output/cs336/lecture_01/02_Phase2_Educator.md`：核心冲突、生产线隐喻、aha moment 和叙事线。

后续如果进入问答或 debug：

- Agent A 只负责公式、shape、memory、compute、接口不变量的诊断。
- Agent B 只负责把诊断转成可理解的问题引导，不覆盖 Phase 1 的数学定义。